

Presented to the College of Computer Studies

De La Salle University - Manila

Term 1, A.Y. 2026-2027

In partial fulfillment of the course

In CEPARCO S01

AVX2 kernels for 3D N-body simulation

Group No. 1

Submitted by:

Francis Billones

Raine Dollentas

Submitted to:

Roger Luis Uy

October 19, 2026

Case Project Proposal

Abstract

We implement a 3D gravitational n-body simulation with collision and merging, whose force and collision kernels are hand-written in x86-64 AVX2/FMA assembly. The all-pairs force computation, which costs quadratic time per step, is vectorized in a sequence of measured stages. First, we translate directly to AVX2, then add fused multiply-add (FMA) instructions, and finally an 8x8 register-tiling scheme with lane rotation adapted from prior AVX-512 work. Collision detection and time integration are also vectorized. Each stage is benchmarked against a scalar C baseline. Because merging changes the particle count over time, performance is reported as interactions per second, which keeps results independent of the number of merges. We also implement a double-precision reference to check for numerical correctness.

1. Description of the Proposed Project

The n-body problem models how particles move under their mutual gravitational attraction. In the direct all-pairs method, each particle's acceleration is the sum of the forces from every other particle, so each time step costs O(N^2) pair interactions for both the force and collision checks. This project implements a 3D gravitational n-body simulation using leapfrog integration, plummer-softened gravity, and merging of overlapping bodies. The force, collision, and integration kernels are hand-written in x86-64 AVX2/FMA assembly and compared against a scalar C baseline. The direct n-body problem appears in astrophysical simulations and molecular dynamics [2], and its O(N^2) cost makes it a classic SIMD target so the project measures how much speedup hand-written assembly gives over scalar C.

1a. Inputs

- N: Number of particles
- Seeded random initial conditions (positions, velocities, masses)
- Simulation parameters:
    - G: Gravitational constant,
    - E: Softening length,
    - DT: Time step,
    - T: Number of steps

1b. Proposed Process

The simulation advances in discrete time steps using leapfrog (kick-drift-kick) integration.

Steps:

1. Compute the acceleration of every particle as affected by all others
2. Update velocities by half a step and positions by a full step
3. Detect overlapping bodies and merges them, conserving mass and momentum
4. Recompute accelerations for the updated set of bodies
5. Update velocities again by half a step

The acceleration of particle i uses Plummer-softened gravity (Aarseth, 2003):

$$a_i = G\sum_j m_j \frac{r_j - r_i}{(|r_j - r_i|^2 + E^2)^{1.5}}$$

The softening term E prevents the denominator from going to 0, which keeps forces finite when the distances get close. It also makes self-interaction exactly zero, meaning the inner loop can be branchless.

i. Processes to be parallelized

1. **Force computation**

   In a naive serial implementation, the program would take one particle and compute its interaction with every other particle one at a time. Each interaction is independent of the others, so they can be computed simultaneously. We can compute particle interactions in parallel, and combine at the end to give that particle's total acceleration.

   To make this possible, the particle data is stored in a Structure of Arrays format (all x-positions together, all y-positions together, and so on). This way, consecutive values of the same property can be loaded into a packed register at once.

2. **Collision detection**

   In a serial version, each particle is checked against every other particle one at a time. This operation is independent, so we can do parallel checks.

3. **Integration**

   After forces are computed, every particle's velocity and position are updated using the same formula, independently of every other particle. We can once again do this multiple particles at a time.

ii. Related works

1. The GitHub repository jdvillal/SIMD_examples contains the source code from a video that served as the inspiration for this project. They implement the same type of simulation (gravity with merging) in Rust, as scalar, AVX2-intrinsic and AVX-512 intrinsic versions. It is 2D, and uses Newton's 3rd law to compute each pair once. Its AVX2 force loop computes the inverse distance with separate square-root and division instructions and loads one target particle at a time.

Our project extends the simulation to 3D and uses hand-written assembly rather than compiler intrinsics.

2. Pedregosa-Gutierrez and Dempsey (2021) optimized a 3D all-pairs direct n-body code with AVX-512 intrinsics in double precision. They identified the inverse square root as the main bottleneck, and used an 8x8 tiling scheme with lane rotation to reuse loaded data. We adapt their tiling scheme to single-precision AVX2.

iii. Parallelization plan

1. Write the scalar C version to serve as a baseline of the full simulation using a structure-of-arrays layout.
2. Write a double-precision scalar version of the force computation, used only to measure the error of each faster kernel.
3. Implement the AVX2 force kernel in NASM, called from C.
    a. Naive AVX2: one target particle is broadcast across all eight lanes. Eight source particles are loaded per 256-bit register. The inverse-distance term is computed with vsqrtps and vdivps. The last N mod 8 particles are handled with masked loads.
    b. AVX2 + FMA: multiply-add chains are fused with vfmadd231ps, reducing instruction count.
    c. AVX2 + FMA + 8x8 tiling: we adapt Pedregosa-Gutierrez and Dempsey (2021)'s 8x8 tiling scheme to reduce loads.
4. Implement the AVX2 collision detection kernel in NASM, called from C.
5. Implement the AVX2 integration kernel in NASM, called from C.

6. Benchmark all versions with no rendering across a range of N. Because collision detection results in merging, the simulation runs faster as time goes on, so we will use interactions per second as our performance metric.

iv. Processes not parallelized

1. Generation of initial conditions
    - This runs only once before the simulation starts, so its cost is irrelevant
2. Resolving each merge (combining the two particles and removing one from the arrays)
    - Merges are rare compared to the number of pair checks, and each merge only involves two particles. It also changes the arrays' layout, which must happen one at a time.
3. Rendering and user interface
    - Drawing the particles is handled by a graphics library and the GPU. It is excluded from all timing measurements

1c. Outputs

1. An interactive 3D visualization of the simulation
    - Buttons to toggle between the different kernels
2. Execution-time comparison between the sequential and SIMD versions, with interactions/second for each kernel across values of N
3. Accuracy comparison with per-step force error relative to the double-precision reference and momentum conservation

2. Target Implementation Platform

- **Hardware:** x86-64 CPU supporting AVX2+FMA3
- **OS / environment:** *Linux, Google Cloud Platform*
- **Languages & tools:** *C, NASM, GCC, raylib*

- **Repository:** https://github.com/francisdbillones/n-body

3. Mid-Project Milestones

Milestone #2 (Week 10)

- *Scalar C simulation in 3D with merging, plus rendering and headless benchmark mode*
- *Double-precision reference*
- *AVX2 force kernel*
- *First benchmark results against the scalar baseline*

Milestone #3 (Week 13)

- *AVX2 collision detection and integration kernels*
- *Complete benchmark and accuracy results*
- *Final demo*

Week	Target	Owner
6	Proposal submitted	Francis Billones, Raine Dollentas
7–8	Scalar C baseline & C raylib visualization	Francis Billones, Raine Dollentas
9–10	AVX2 force kernel & double-precision reference (Milestone 2)	Francis Billones
11–12	AVX2 collision detection & integration kernels	Raine Dollentas
13	Final demo + repo	Francis Billones, Raine Dollentas

4. References

[1] jdvillal. (n.d.). *SIMD_examples* [Computer software]. GitHub. https://github.com/jdvillal/SIMD_examples

[2] Pedregosa-Gutierrez, J., & Dempsey, J. (2021). *Direct N-body problem optimisation using the AVX-512 instruction set*(arXiv:2106.11143). arXiv. https://doi.org/10.48550/arXiv.2106.11143

[3] Aarseth, S. J. (2003). *Gravitational N-body simulations: Tools and algorithms*. Cambridge University Press.